BIPES Block DSL Guide

This document summarizes what the Block DSL does today in this repository, how the generation pipeline works, what each script is responsible for, the main rules, and how blocks are annotated in Python source files.

What The DSL Does

The Block DSL turns annotated Python libraries into Blockly artifacts. A Python library file is treated as the source of truth. The DSL reads `# @block` annotations, extracts Python signatures and defaults, validates the metadata, builds an internal block model, and emits three generated files per target library:

- Blockly block definitions in `static/page/blocks/blocks/*_dsl.js`
- Blockly Python generators in `static/page/blocks/pythonic/*_dsl.js`
- Toolbox definitions in `templates/page/blocks/definitions/*_dsl.md`

> The DSL is explicit, not automatic. A library only participates when it is listed in `server/block_dsl/integrate.py`.

How Generation Is Triggered

The normal manual entrypoint is:

```
python3 scripts/generate_block_dsl.py
```

That script imports `generate_default_artifacts` from `server.block_dsl`. The integration layer reads the hardcoded targets, runs the pipeline for each source file, and writes the generated artifacts only when the content has changed.

Pipeline Overview

1. **1. Scan annotations**: CommentAnnotationScanner finds # @block blocks, parses JSON, and captures tooltip comment lines.
2. **2. Extract Python structure**: PythonAstSourceExtractor parses the file with ast and builds ModuleSource, ClassBlockSource, FunctionBlockSource, and ParameterSpec objects.
3. **3. Resolve metadata**: SourceMetadataResolver attaches annotations to classes/methods/functions and merges class defaults into method metadata.
4. **4. Validate rules**: ModelValidator checks the resolved model for invalid keys, bad param config, label issues, and invalid instance configuration.
5. **5. Build block model**: BlockModelBuilder creates BlockSpec and InputSpec objects, orders inputs, injects ID selectors, and chooses generated block names.
6. **6. Build toolbox model**: SimpleToolboxBuilder groups blocks by category.
7. **7. Build Python generator templates**: PythonGeneratorBuilder creates the code template that each Blockly block should generate.
8. **8. Emit files**: emitters.py writes the Blockly block JS, Blockly Python generator JS, and toolbox definition markdown for each configured target.

In code, this orchestration happens in `server/block_dsl/pipeline.py` via `DefaultPipeline.parse(...)`.

What Each Script Does

File	Responsibility
`scripts/`	Manual entrypoint. Calls generate_default_artifacts(ROOT) to regenerate all configured DSL

`generate_block_dsl.py`	outputs.
`server/block_dsl/integrate.py`	Hardcoded target registry. Chooses which Python library files are DSL-managed and where each generated artifact is written.
`server/block_dsl/pipeline.py`	Wires the pipeline together: scan annotations, extract AST, resolve metadata, validate, build blocks, build toolbox, build Python generator templates.
`server/block_dsl/scanner.py`	Reads # @block annotations from comments, supports one-line and multi-line JSON, and collects tooltip comment lines.
`server/block_dsl/extractor.py`	Parses Python source with ast and extracts classes, functions, methods, parameters, defaults, and type hints into the source model.
`server/block_dsl/metadata.py`	Converts raw annotation JSON into typed Metadata objects and merges class metadata with method metadata.
`server/block_dsl/resolver.py`	Attaches scanned annotations to extracted source objects and decides which functions/methods are public blocks.
`server/block_dsl/validation.py`	Applies DSL rules and rejects invalid metadata combinations, bad params config, and invalid multiple-instance definitions.
`server/block_dsl/model.py`	Defines the internal dataclasses and enums used by the DSL: Metadata, ParameterSpec, InputSpec, BlockSpec, ParseResult, and more.
`server/block_dsl/builder.py`	Transforms resolved source objects into final BlockSpec objects, humanizes names, infers input types, and injects multiple-instance ID inputs.
`server/block_dsl/python_generator.py`	Builds Python code templates from BlockSpec objects for both singleton and multiple-instance blocks.
`server/block_dsl/toolbox_builder.py`	Groups generated blocks into toolbox categories.
`server/block_dsl/emitters.py`	Emits the final generated artifacts: Blockly block JS, Blockly Python generator JS, and toolbox definition markdown/XML fragments.
`server/block_dsl/errors.py`	Defines the DSL-specific exception types used during scanning and validation.
`server/block_dsl/__init__.py`	Package export layer for the DSL integration entrypoints.

Current Hardcoded Targets

Python Source	Category	Generated Outputs

`PicoRobotics.py`	Robotics Board	`static/page/blocks/blocks/roboticsboard_dsl.js`
		`static/page/blocks/pythonic/roboticsboard_dsl.js`
		`templates/page/blocks/definitions/roboticsboard_dsl.md`

`sand_table_robot.py`	Sand Drawing Machine	`static/page/blocks/blocks/sand_table_robot_dsl.js`
		`static/page/blocks/pythonic/sand_table_robot_dsl.js`
		`templates/page/blocks/definitions/sand_table_robot_dsl.md`

`ds1302.py`	DS1302	`static/page/blocks/blocks/ds1302_dsl.js`
		`static/page/blocks/pythonic/ds1302_dsl.js`
		`templates/page/blocks/definitions/ds1302_dsl.md`

`dfplayer.py`	DFPlayer	`static/page/blocks/blocks/dfplayer_dsl.js` `static/page/blocks/pythonic/dfplayer_dsl.js`

		`templates/page/blocks/definitions/dfplayer_dsl.md`

`st7735s.py`	ST7735S	`static/page/blocks/blocks/st7735s_dsl.js`
		`static/page/blocks/pythonic/st7735s_dsl.js`
		`templates/page/blocks/definitions/st7735s_dsl.md`

Current DSL Rules

- Annotations use # @block { ...json... } immediately above a class or def.
- JSON may be one-line or multi-line commented JSON.
- Comment lines between # @block and the next class/def become the tooltip.
- Only annotated functions and methods become public blocks.
- Annotated __init__ methods become constructor blocks.
- Class metadata provides defaults; method metadata overrides class metadata.
- Unknown metadata keys are rejected.
- Supported param input kinds are input_value, field_dropdown, field_input, and variable.
- field_dropdown requires options.
- Python default parameter values become default block values unless params.<name>.defaultValue overrides them.
- Multiple-instance classes are normalized to numeric id inputs.
- For multiple-instance blocks, ID # is always the first input.
- Single pin parameters get pinout shadows; pins lists get list shadows made of pinout blocks.
- Block titles are generated from names: CamelCase split, underscores/hyphens become spaces, and common hardware abbreviations like SDA/SCL/CLK/RST stay uppercase.
- Generated Python for singleton blocks uses a fixed instance name, usually board.
- Generated Python for multiple-instance blocks uses a registry dictionary keyed by numeric id.

These rules match the current implementation documented in `BLOCK_DSL_RULES.md` and enforced in the DSL code.

How To Annotate Blocks

Basic Pattern

Place `# @block { ...json... }` directly above a class or function. If you want a tooltip, place plain comment lines after the annotation and before the next `class` or `def`. Those lines become the block tooltip.

```
# @block {
#   "category": "Robotics Board",
#   "color": 180,
#   "instanceMode": "singleton",
#   "instanceName": "board",
#   "methodInstanceMode": "fixed_name"
# }
# Create and control the robotics board.
class KitronikPicoRobotics:

    # @block {}
    # Create the robotics board instance.
    def __init__(self, I2CAddress=108, sda=8, scl=9):
        ...

    # @block {
    #   "params": {
    #     "direction": {
    #       "inputKind": "field_dropdown",
    #       "options": [["Forward", "f"], ["Reverse", "r"]]
    #     }
    #   }
    # }
```

```
    # Run a motor.
    def motorOn(self, motor, direction, speed):
        ...
```

Multiple-Instance Pattern

For multiple-instance classes, current behavior normalizes the instance selector to a numeric `id`. Methods get `ID #` as the first input and generated Python uses a registry dictionary keyed by that id.

```
# @block {
#   "category": "Sand Drawing Machine",
#   "instanceMode": "multiple",
#   "methodInstanceMode": "key_input"
# }
# Control multiple sand table robots.
class SandTableRobot:

    # @block {}
    # Create a robot instance.
    def __init__(self, id=1, motor1_pins=(17,16,15,14), motor2_pins=(21,20,19,18)):
        ...

    # @block {}
    # Return the robot to the home position.
    def home(self):
        ...
```

Params Overrides

Inside a function or method annotation, `params` lets you change how specific parameters are rendered. Common uses are dropdowns, variable fields, type checks, and explicit default values.

```
# @block {
#   "params": {
#     "direction": {
#       "inputKind": "field_dropdown",
#       "options": [["Forward", "f"], ["Reverse", "r"]]
#     },
#     "spi": {
#       "inputKind": "variable"
#     }
#   }
# }
```

Authoring Guidelines

- Annotate only the methods you want exposed as blocks.
- Annotate `__init__` when you want a constructor block.
- Use normal Python defaults in function signatures whenever possible.
- Use names like `pin` or `pins` when you want the built-in pin shadow behavior.
- Use `params` for UI-specific overrides such as dropdowns or variable fields.
- Keep comments directly under the annotation if you want them to become tooltips.

Current Limitations

- Visible block titles are generated from names, even though `label` metadata is still parsed and validated.

- Multiple-instance handling is standardized around numeric `id`.
- Type inference is intentionally simple.
- Complex runtime objects may still need explicit `params` configuration.
- The DSL works from signatures and metadata, not by inferring semantics from function bodies.

Useful Files To Read Next

- `BLOCK_DSL_RULES.md` for the current rule set
- `server/block_dsl/pipeline.py` for the high-level orchestration
- `server/block_dsl/integrate.py` for the active source files and output paths
- `server/block_dsl/builder.py` and `server/block_dsl/emitters.py` for the final block shaping and file emission